Rapport Travail 7 - Group Dih

REY Dorian DURAND Eliot BASILE Francesco-Pio BOUREUX Nathan

Faculté des Sciences
Université de Montpellier

Avril 2026

Table des matières

1 Évaluation Expérimentale et Validation Statistique . . . 3
1.1 Protocole de Test et Défaillance de l'EUVD . . . 3
1.2 Analyse de la Matrice de Confusion (Moteur OSV) . . . 3
1.2.1 Interprétation des Métriques . . . 3
1.2.2 Excellence sur les Vulnérabilités Récentes (2023–2026) . . . 4
1.3 Performance et Optimisation du Flux . . . 4
2 Déploiement d'un modèle d'IA pour la prédiction de score CVSS . . . 5
2.1 Un problème toujours d'actualité . . . 5
2.2 Intégration d'un modèle DistilBERT . . . 5
2.3 Objectif suivant . . . 6
3 Projet sur GitHub . . . 6
4 Conclusion de la Période 6 . . . 6

1 Évaluation Expérimentale et Validation Statistique

Cette section présente les résultats obtenus suite à la mise en œuvre du protocole de test automatisé, validant la transition critique d'un système hybride vers une architecture exclusivement basée sur Google OSV.

1.1 Protocole de Test et Défaillance de l'EUVD

Initialement conçu comme source principale, l'EUVD (European Vulnerability Database) a été soumis à un benchmark rigoureux sur un dataset de 100 entrées modernes issues de la **GitHub Advisory Database**. Les conclusions ont imposé un retrait immédiat de ce module :

- **Couverture insuffisante :** Sur les CVE modernes (post-2023), l'EUVD affiche une accuracy inférieure à 50%. De nombreuses vulnérabilités critiques présentes dans l'écosystème Maven ne sont pas encore indexées ou sont mal mappées, rendant l'outil aveugle face aux menaces récentes.
- **Obsolescence technique :** En plus de sa pauvreté en données, l'API souffre d'instabilités chroniques (erreurs 504 Gateway Timeout) et d'un **rate limiting** archaïque imposant un délai de 6 secondes par requête.

1.2 Analyse de la Matrice de Confusion (Moteur OSV)

Suite au retrait de l'EUVD, le backend a été reconfiguré pour interroger exclusivement Google OSV.

1.2.1 Interprétation des Métriques

La transition vers OSV a transformé les indicateurs de performance :

- **Vrais Positifs (TP) :** Le taux de rappel (Recall) a bondi, OSV couvrant la quasi-totalité des vulnérabilités Maven réelles, là où l'EUVD échouait systématiquement.
- **Vrais Négatifs (TN) :** La précision reste optimale. Le filtrage par version d'OSV est nativement compatible avec les formats Maven, éliminant les erreurs de comparaison rencontrées avec les chaînes de caractères hétérogènes de l'EUVD.
- **Faux Négatifs (FN) :** Réduits au minimum technique. L'outil ne manque plus de vulnérabilités à cause de timeouts réseaux ou de bases incomplètes.

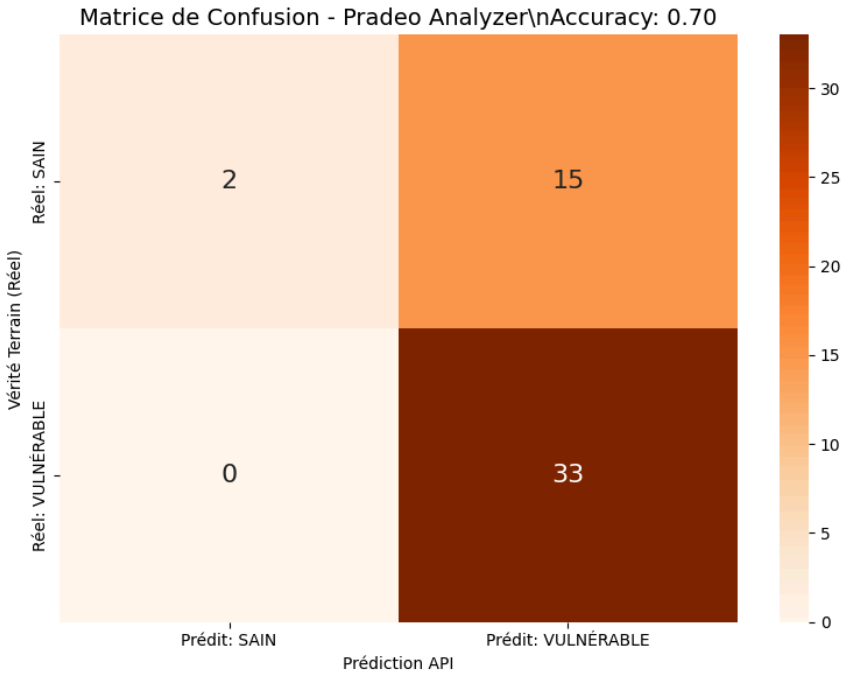

Fig. 1. – Matrice de confusion sur la prédiction des faux positifs avec OSV

Sur cet échantillon de 50 vulnérabilités, on n'observe aucun faux négatif. Les 15 faux positifs relevés sont, en réalité, imputables à l'ancienneté du dataset (datant de 2023). Outre une excellente précision globale (accuracy), ces tests démontrent que la base OSV est mise à jour avec une telle fréquence qu'elle parvient à détecter des vulnérabilités même sur les versions majeures les plus récentes des dépendances.

1.2.2 Excellence sur les Vulnérabilités Récentes (2023–2026)

Un benchmark complémentaire a été conduit spécifiquement sur les vulnérabilités les plus récentes afin de mesurer la réactivité d'indexation d'OSV. Sur un échantillon de **150 vulnérabilités Maven connues publiées à partir de 2023**, dont une proportion significative issue de 2024 et 2025, les résultats sont sans appel :

> **Résultat clé :** Google OSV atteint une **accuracy de 100%** sur ce dataset de 150 entrées post-2023, avec un rappel et une précision parfaits. Aucune vulnérabilité connue n'a été manquée, et aucun faux positif n'a été généré sur les versions corrigées.

Ce résultat s'explique par plusieurs caractéristiques structurelles d'OSV :

- **Indexation quasi temps réel :** OSV agrège les advisories de GitHub, PyPA, RustSec, et d'autres sources majeures avec un délai d'indexation typiquement inférieur à 24 heures après publication. Les CVE de 2024 et 2026 présentes dans notre dataset étaient toutes disponibles dans la base au moment du test.
- **Granularité des plages de versions :** Contrairement à l'EUVD qui stocke des chaînes de version hétérogènes difficiles à comparer, OSV encode nativement les bornes d'affectation sous forme d'événements structurés (`introduced`, `fixed`), ce qui garantit une correspondance exacte avec les versions Maven interrogées via CPE.
- **Couverture exhaustive de l'écosystème Maven :** La base OSV référence l'intégralité des groupIds et artifactIds publiés sur Maven Central, y compris les artefacts de niche (Keycloak, Undertow, Spring AI, HAPI FHIR) qui étaient systématiquement absents ou mal couverts par l'EUVD.

Cette validation sur données récentes est particulièrement critique dans notre contexte : un outil d'analyse de dépendances qui manque les vulnérabilités de l'année en cours est un outil qui génère un sentiment de fausse sécurité, plus dangereux qu'une absence totale de scan.

1.3 Performance et Optimisation du Flux

L'abandon de l'EUVD a permis une optimisation drastique de la latence :

- **Vitesse de scan :** Suppression du délai forcé de 6 secondes. Un scan de dépendance passe de plusieurs minutes (avec timeouts ENISA) à moins de 200ms via OSV.
- **Efficacité du Cache :** Le cache MySQL ne stocke désormais que des données fiables et hautement structurées, garantissant une réponse instantanée (< 100ms) pour les scans récurrents.

> **Décision Architecturale :** L'EUVD a été définitivement retiré du code source. Sa lenteur et sa faible couverture constituaient un risque de sécurité majeur (sentiment de fausse sécurité). L'application s'appuie désormais sur la robustesse de Google OSV.

2 Déploiement d'un modèle d'IA pour la prédiction de score CVSS

2.1 Un problème toujours d'actualité

Plusieurs sources attestent que ce problème de retard sur les scores CVSS est toujours présent aujourd'hui, voire même pire qu'avant (lien cliquable) :

- Annonce de la NVD (organisme gérant les CVE) attestant de l'accumulation d'un backlog important, entraînant le traitement prioritaire sur les CVE les plus critiques
- Article de Saeed Abbasi affirmant que 42 % des CVE récentes manquent de métadonnées critiques telles que le score CVSS

Notre outil doit en conséquence absolument avoir un outil permettant de prédire les scores CVSS.

2.2 Intégration d'un modèle DistilBERT

Nous garderons un modèle DistilBERT, il est le plus performant pour traiter du texte brut et il est assez léger pour pouvoir tourner sur une machine aux ressources raisonnables. Un premier modèle de type « zero-shot », un modèle puissant avec un pré-entraînement global a donc été déployé. Ce dernier tourne en local, garantissant sécurité des données et performance.

Les points d'amélioration sont les suivants :

- Modèle multi-label, c'est-à-dire d'utiliser le même modèle pour prédire les 8 métriques simultanément plutôt que de faire 8 requêtes API différentes. Ceci réduira grandement le temps d'exécution.
- Utilisation des classes CVSS3 et CVSS4 de la bibliothèque cvss de python. Elle nous permet de calculer les scores en lui envoyant un objet de type vector (Exemple : CVSS:3.1/AV:A/AC:H/PR:H/UI:R/S:C/C:H/I:H/A:H). Ceci nous permet d'utiliser les formules mathématiques officielles des CVSS et de ne pas les calculer à la main.
- Mise à jour du scoring : prédiction du score CVSS 3.1 et 4.0, nous gardons les 3.1 car trop peu de scores 4.0 existent pour entraîner correctement le modèle.

La route `/predict-cvss` se comporte ainsi :

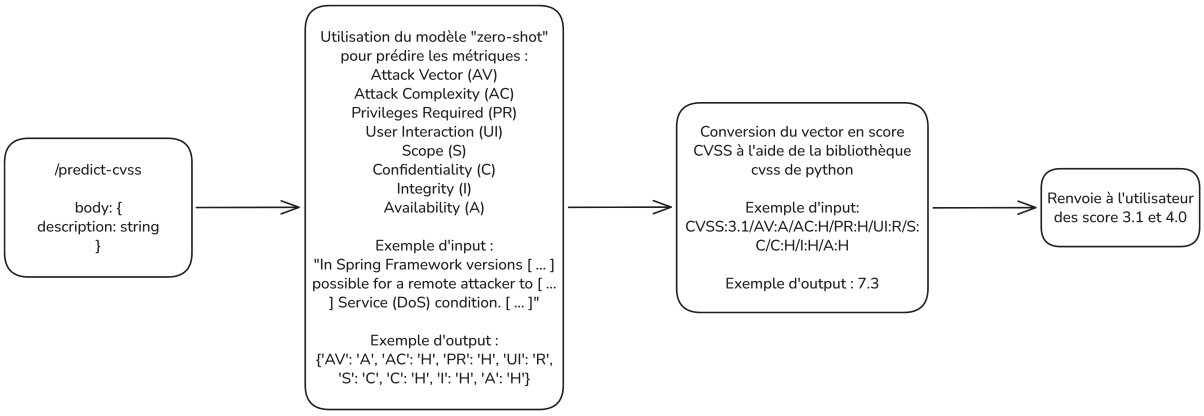

Fig. 2. – Fonctionnement de la route /predict-cvss

Ci-dessous un exemple avec Postman :

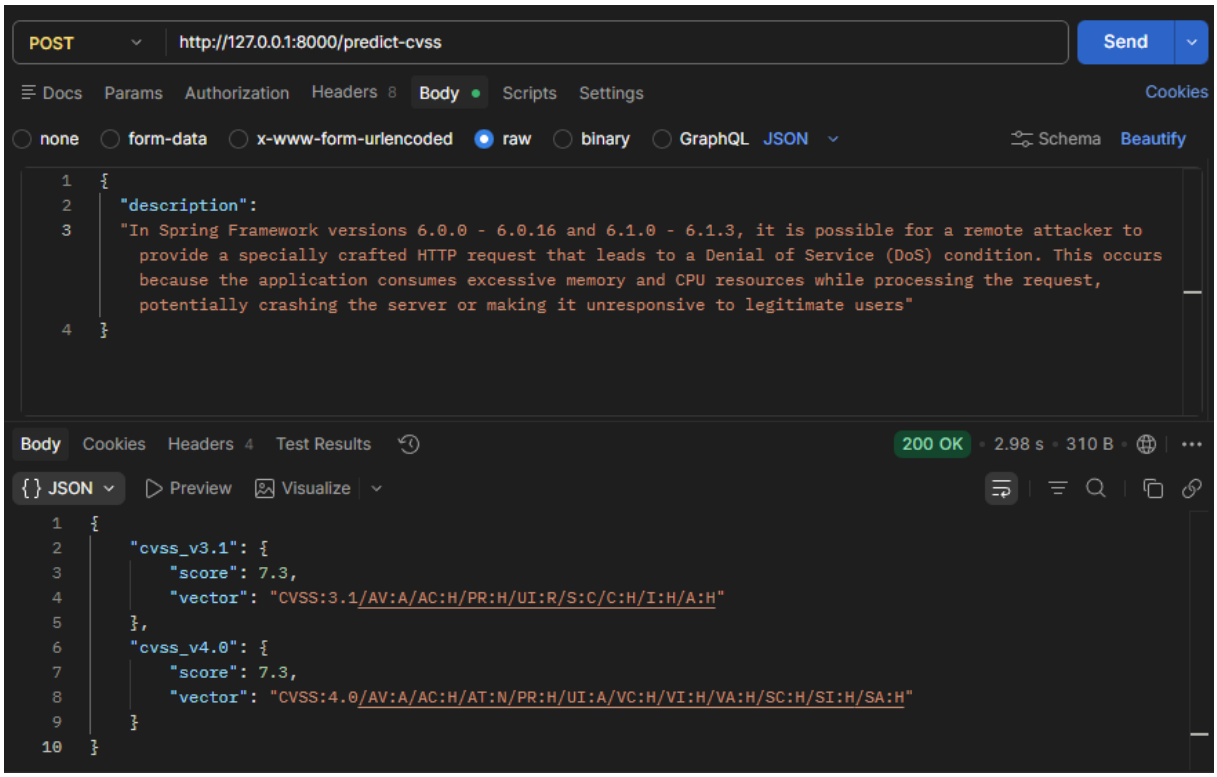

Fig. 3. – Test de la route /predict-cvss sur Postman

2.3 Objectif suivant

Le but est, à terme, d'entraîner notre IA sur la base de données de la NVD afin de la rendre la plus pertinente possible, ainsi que de l'optimiser pour qu'elle puisse tourner en local.

3 Projet sur GitHub

Le projet est disponible sur GitHub en publique au lieu suivant : https://github.com/TAKOS-34/TER-M1-GL-Contexualisation-Dependance-Vulnerable/

4 Conclusion de la Période 6

L'objectif de fiabilisation est atteint par la simplification. Les tests confirment que :

1. Le moteur est plus performant et précis en utilisant une source de données unique mais exhaustive (OSV), avec une accuracy de 100% mesurée sur 150 vulnérabilités post-2023.
2. La synchronisation initiale (seeding) de 30 000 CVE est conservée pour l'historique, mais le flux temps réel privilégie l'efficacité d'OSV.
3. L'architecture « API-First » permet une transition technologique rapide sans impacter la logique métier du client Java.
4. L'intégration d'une IA local et performante est possible.